

รอบ 5 — ตรวจสอบความน่าเชื่อถือของผล Cross-Dataset

เอกสารนี้ตอบรายการตรวจสอบที่อาจารย์ขอ ณ วันที่ 28 กันยายน 2569

1. ผลตรวจ AUC ให้ครบ ทั้ง 5 seeds, duplicate/near-duplicate, Logistic Regression และ label-shuffle
2. Control ภายใน dataset เดียวกัน ว่าคนละ file/time batch แยกกันได้ง่ายหรือไม่
3. เปรียบเทียบ Dataset Importance กับ Attack Importance ว่า feature ใดเป็น fingerprint อย่างเดียว และ feature ใดใช้จำแนก attack ด้วย
4. สรุปผล Cross-Dataset ใหม่หลังแก้ Clean-before-Sampling ว่า asymmetry และผลของ split mode ยังอยู่หรือเปลี่ยนไปอย่างไร

ทุกตัวเลขในเอกสารนี้อ่านมาจากไฟล์ผลลัพธ์จริงในโฟลเดอร์ data/ ข้างเคียง ซึ่งเป็นสำเนาของ results/crossdataset/ และสร้างซ้ำได้ด้วยสคริปต์ที่ระบุไว้ท้ายเอกสาร

สรุปสั้น

ข้อ 1 AUC ระหว่าง 2017 กับ 2018 สูงจริงและไม่ได้มาจากข้อผิดพลาดของ pipeline ทั้ง 5 seeds ให้ค่าเกือบเท่ากัน ไม่มีแถวข้ามคร่อม train/test เลยใน 30 การสุ่ม Logistic Regression ก็แยกได้ 0.9478 ขึ้นไป และเมื่อสลับ label ทิ้ง AUC ตกลงมาอยู่ที่ 0.49 ถึง 0.51 ทุกโมเดล ทุกคลาส

ข้อ 2 คนละ file หรือคนละช่วงเวลาภายใน dataset เดียวกันแยกได้ แต่ไม่แยกเท่าข้าม dataset ภายใน dataset ได้ 0.66 ถึง 0.94 ข้าม dataset ได้ 0.95 ถึง 1.00 ทิศทางนี้ตรงกันทั้ง 7 โมเดล ส่วน DoS กับ DDoS ที่ดูเหมือนแยกได้ 1.0 ภายใน dataset นั้น ตรวจแล้วเป็นเพราะคลัง 2018 วางเครื่องมือโจมตีแต่ละตัวไว้คนละวัน ไม่ใช่เพราะวันต่างกัน

ข้อ 3 feature ที่บอกว่า record มาจาก dataset ไหน กับ feature ที่ใช้จำแนก attack เป็นชุดเดียวกันเป็นส่วนใหญ่ rank correlation ระหว่างสองอย่างอยู่ที่ +0.72 ถึง +0.77 และมี feature เพียงตัวเดียวที่เป็น fingerprint อย่างเดียวสำหรับทั้งสองโมเดลที่วัด

ข้อ 4 asymmetry ยังอยู่ครบทั้ง 7 โมเดล แต่เล็กลงจาก 40.4 เป็น 33.6 จุด และการแบ่งแบบสุ่ม ยังซ่อน asymmetry ไว้เหมือนเดิม โมเดลที่ดีที่สุดในทิศ 2018 ไป 2017 เปลี่ยนจาก stacking เป็น logistic regression

1. ผลตรวจ AUC ครบทุกด้าน

1.1 สิ่งที่เปลี่ยนจากรอบก่อน

สคริปต์ตรวจเดิมรัน model ladder, near-duplicate และ null control แค่ seed 42 ตัวเดียว แล้วรายงานผลนั้นเหมือนเป็นคำตอบ และ ยังไม่มี label-shuffle เลย รอบนี้แก้ให้ทุกการตรวจที่ขึ้นกับการสุ่มรันครบทั้ง 5 seeds และเพิ่ม label-shuffle

label-shuffle กับ null control เป็นคนละการทดสอบกัน

- null control เอา dataset เดียวมาแบ่งครึ่งแบบสุ่ม แล้วดูว่าวิธีการสร้างการแยกขึ้นมาเองได้ไหม
- label-shuffle ใช้แถวจริงชุดเดียวกับผลหลักทุกแถว แต่สลับ label 2017/2018 ทิ้งก่อนแบ่งข้อมูล ถ้ายังได้ AUC สูง แปลว่า pipeline รู้ label ผ่านช่องทางอื่นที่ไม่ใช่ feature

1.2 AUC หลักทั้ง 5 seeds

LightGBM ตัวเดียวกับที่ใช้ทำ adversarial validation รันซ้ำแล้วได้ค่าเท่ากับรอบตรวจก่อนทุกตำแหน่ง ขนาดตัวอย่างสูงสุด 8,000 แถวต่อฝั่ง

คลาส	n ต่อฝั่ง	seed 42 ถึง 46	เฉลี่ย $\pm$ sd
Benign	8,000	0.9990 0.9991 0.9993 0.9990 0.9988	0.9990 $\pm$ 0.0002
DDoS	8,000	1.0000 ทุก seed	1.0000 $\pm$ 0.0000
DoS	5,132	1.0000 ทุก seed	1.0000 $\pm$ 0.0000
Brute Force	2,459	1.0000 0.9999 1.0000 1.0000 1.0000	1.0000 $\pm$ 0.0000
Bot	1,948	1.0000 ทุก seed	1.0000 $\pm$ 0.0000
Web Attack	859	1.0000 ทุก seed	1.0000 $\pm$ 0.0000

Infiltration ทดสอบไม่ได้ เพราะคลัง 2017 มี flow ประเภทนี้เพียง 36 แฉ

1.3 Duplicate และ near-duplicate

near-duplicate คือแฉ test ที่อยู่ใกล้แฉ train มากกว่าระยะที่แฉ train อยู่ห่างกันเองถึง 99.9% ของกรณี
เกณฑ์นี้ตั้งมาจากข้อมูลเอง ระดับที่เกิดขึ้นได้โดยบังเอิญจึงเท่ากับ 0.10% พอดี

คลาส	แฉซ้ำเป๊ะคร่อม split (5 seeds)	near-dup เฉลี่ย	near-dup สูงสุด
Benign	0 0 0 0 0	0.10%	0.15%
DoS	0 0 0 0 0	0.16%	0.29%
Brute Force	0 0 0 0 0	0.20%	0.41%
Bot	0 0 0 0 0	0.17%	0.34%
DDoS	0 0 0 0 0	6.82%	7.17%
Web Attack	0 0 0 0 0	2.75%	4.07%

ไม่มีแฉซ้ำเป๊ะคร่อม train/test เลยสักแฉในการสุ่มทั้ง 30 ครั้ง DDoS กับ Web Attack มี near-duplicate สูงกว่าระดับบังเอิญจริงในทุก seed แต่ไม่ได้อธิบายค่า AUC เพราะ decision stump
ซึ่งใช้เกณฑ์เดียวบนคอลัมน์เดียวและจำแฉไม่ได้ ก็ยังแยกสองคลาสนี้ได้ 0.99 ขึ้นไป

1.4 Model ladder รวม Logistic Regression

ความจุของโมเดลจากน้อยไปมาก ค่าเฉลี่ย 5 seeds

คลาส	stump	tree ลึก 3	LR	LR (rank)	LightGBM
Benign	0.7089	0.9490	0.9478	0.9803	0.9990
DDoS	0.9917	0.9999	1.0000	1.0000	1.0000
DoS	0.9370	0.9858	0.9951	0.9998	1.0000
Brute Force	0.9950	0.9988	0.9999	1.0000	1.0000
Bot	0.9959	1.0000	0.9990	1.0000	1.0000
Web Attack	0.9992	0.9992	1.0000	1.0000	1.0000

LR ของ Benign ที่เคยรายงานไว้ 0.9531 เป็นค่าของ seed 42 ตัวเดียว ค่าเฉลี่ย 5 seeds คือ

0.9478 $\pm$ 0.0066

LR (rank) คือ logistic regression หลังแปลงแต่ละคอลัมน์เป็นลำดับ เหลือไว้แค่ลำดับค่า ใช้เพราะ flow feature กระจายตัวกว้างหลายหลักและเบ้มาก

1.5 Label-shuffle และ null control

การทดสอบ	ช่วง AUC ทุกโมเดล ทุกคลาส
label-shuffle (แถวจริง สลับ label)	0.4928 ถึง 0.5055
null control (dataset เดียว แบ่งครึ่งสุ่ม ครึ่งใหม่ทุก seed)	0.4915 ถึง 0.5273

1.6 สรุปข้อ 1

- pipeline ไม่มีทางรู้ label นอกจากผ่าน feature เพราะ label-shuffle ตกลงที่ระดับสุ่มทุกกรณี
- ความขงของโมเดลไม่ใช่ค่าอธิบาย เพราะ logistic regression และ stump ก็แยกได้
- ไม่มีการจำแถว เพราะไม่มีแถวซ้ำคร่อม split เลย

2. Control ภายใน dataset เดียวกัน

2.1 ทำไมต้องทำ

null control ในข้อ 1 พิสูจน์ได้แค่ *วิธีการ* ไม่สร้างการแยกขึ้นมาเอง แต่พิสูจน์ *ข้อสรุป* ไม่ได้ เพราะการแบ่งแบบสุ่มคลุกทุกวันก็เก็บข้อมูลเข้าไปทั้งสองฝั่ง ความต่างที่ผูกกับช่วงเวลา ที่เก็บข้อมูลจึงรอดไม่ได้ตั้งแต่ต้น

control นี้จึงลากเส้นแบ่ง *ภายใน* dataset เดียวแล้วรักษาโครงสร้างไว้ ถ้าคนละวันในคลังเดียวกัน แยกได้ดีพอ ๆ กับ 2017 เทียบ 2018 สิ่งทีเรียกว่า dataset fingerprint ก็เป็นแค่ความต่างตามปกติระหว่างการเก็บข้อมูลแต่ละครั้ง

2.2 วิธี

- ใช้ขั้นตอนเดียวกับ adversarial validation ทุกอย่าง และรันด้วยโมเดลของ protocol ครบทั้ง 7 ตัว กับ stump
- เส้นแบ่งภายใน dataset มีสองแบบ คือครึ่งแรกเทียบกับครึ่งหลังตามลำดับเวลา และ capture file ชุดหนึ่งเทียบกับอีกชุด
- ทุกเส้นแบ่งในคลาสเดียวกันใช้จำนวนแถวต่อฝั่งเท่ากัน เพราะ AUC ขยับตามขนาดตัวอย่าง
- แบ่ง capture file ด้วยขนาด ไม่ใช่เรียงตามวันที่ มิฉะนั้นจะได้การแบ่งตามเวลาซ้ำอีกรอบ
- เขียนกติกาการอ่านผลไว้ก่อนรัน

2.3 ผล

ตัวเลขคือ ข้าม dataset / ภายใน dataset สูงสุด โดย "ภายใน dataset สูงสุด" คือเส้นแบ่งภายในที่ แยกได้ดีที่สุดของคลาสนั้น เพื่อเทียบกับกรณีที่เข้าทางค่าอธิบายฝั่งตรงข้ามมากที่สุด

คลาส	n ต่อฝั่ง	LightGBM	Logistic Regression	stump	ช่วงภายใน dataset ทั้ง 7 โมเดล
Benign	8,000	0.9990 / 0.8058	0.9478 / 0.6580	0.7089 / 0.5845	0.6580 ถึง 0.8112
Brute Force	1,229	1.0000 / 0.8960	0.9989 / 0.8648	0.9973 / 0.8474	0.8606 ถึง 0.8993
Bot	974	1.0000 / 0.9327	0.9999 / 0.7910	0.9945 / 0.7521	0.7584 ถึง 0.9365

คลาส	n ต่อฝั่ง	LightGBM	Logistic Regression	stump	ช่วงภายใน dataset ทั้ง 7 โมเดล
Web Attack	338	1.0000 / 0.8404	0.9941 / 0.7365	0.9990 / 0.6903	0.7095 ถึง 0.8404
DoS	1,341	0.9999 / 1.0000	0.9931 / 0.9985	0.9409 / 0.9478	0.9952 ถึง 1.0000
DDoS	5,271	1.0000 / 1.0000	1.0000 / 1.0000	0.9918 / 0.9981	0.9997 ถึง 1.0000

null control ในรอบนี้อยู่ที่ 0.5311 สูงสุด

คำตอบข้อ 2 สี่คลาสแรกให้ผลตรงกันทุกโมเดล คนละ file หรือคนละช่วงเวลาภายใน dataset เดียวกันแยกได้ แต่ได้เพียง 0.66 ถึง 0.94 ขณะที่ข้าม dataset ได้ 0.95 ถึง 1.00

2.4 DoS และ DDoS ที่ได้ 1.0 ภายใน dataset

เปิดดูว่าสองฝั่งของเส้นแบ่งมีอะไรอยู่ พบว่าไม่ใช่ "วันต่างกัน" แต่เป็นเครื่องมือโจมตีคนละตัว

คลาส	ฝั่งหนึ่ง	อีกฝั่ง
DDoS	20 ก.พ. มีแค่ LOIC-HTTP	21 ก.พ. มีแค่ HOIC และ LOIC-UDP
DoS	16 ก.พ. มีแค่ Hulk และ SlowHTTPTest	15 ก.พ. มีแค่ GoldenEye และ Slowloris

คลัง 2018 วางเครื่องมือแต่ละตัวไว้คนละวัน แล้ว schema 7 คลาสของเราขมมารวมเป็นคลาสเดียว

จึงรันการทดสอบที่ตรงที่ละอย่างเพื่อแยกสองคำอธิบายออกจากกัน

เครื่องอะไร เปลี่ยนอะไร	ตัวอย่าง	AUC	stump
เครื่องมือเดียวกัน คนละวัน	Benign 20 ก.พ. เทียบ 22 ก.พ.	0.8340	0.5429
เครื่องมือเดียวกัน คนละวัน	Infiltration 1 มี.ค. เทียบ 28 ก.พ.	0.7460	0.5917
คนละเครื่องมือ คลาสเดียวกัน	Hulk เทียบ GoldenEye	1.0000	0.9472
คนละเครื่องมือ คลาสเดียวกัน	LOIC-HTTP เทียบ HOIC	1.0000	1.0000

ความต่างระหว่างวันเก็บข้อมูลเมื่อวัดแบบสะอาดอยู่ที่ 0.75 ถึง 0.83 ส่วนค่า 1.0 มาจากการ เป็นคนละเครื่องมือ

2.5 ตรวจสอบอีกชั้น: เครื่องมือเดียวกันข้าม dataset

ตรวจคนเดียวกันย้อนกลับมาที่ผลหลักได้ คำว่า DoS ในสองคลังเป็นเครื่องมือหลายตัวในสัดส่วนต่างกัน

ส่วนหนึ่งของ AUC 1.0 อาจมาจากสัดส่วนนั้น จึงรันโดยตรึงให้เป็นโปรแกรมโจมตีตัวเดียวกัน แล้วเปลี่ยนแค่ dataset

เครื่องมือ	คลาส	n ต่อฝั่ง	เครื่องมือเดียวกัน ข้าม dataset	stump	ทั้งคลาส (n เท่ากัน)
Hulk	DoS	8,000	0.9999	0.9529	1.0000
GoldenEye	DoS	8,000	1.0000	0.8588	1.0000
Slowloris	DoS	5,374	1.0000	0.8614	1.0000
Bot	Bot	1,948	1.0000	0.9945	1.0000
LOIC-HTTP	DDoS	8,000	1.0000	0.9940	1.0000
Web brute force	Web Attack	553	1.0000	0.9982	1.0000

LOIC-HTTP เป็นคู่เดียวที่ระบุตัวตนจากเอกสารของ CIC ไม่ใช่จากการที่ label ทั้งสองฝั่งเขียน ค่าเดียวกัน ส่วน SlowHTTPTest และ XSS ทดสอบไม่ได้เพราะแฉไม่พอ

เครื่องโปรแกรมโจมตีแล้วตัวเลขไม่ขยับเลย สัดส่วนของเครื่องมือจึงไม่ใช่สิ่งที่ตัวจำแนกใช้

2.6 สรุปข้อ 2

เครื่องอะไรไว้	AUC
แบ่งครึ่งสุ่ม ไม่มีเส้นแบ่งจริง	0.49 ถึง 0.53
dataset เดียวกัน เปลี่ยนวันหรือไฟล์	0.66 ถึง 0.94 (สะอาดที่สุด 0.83)
โปรแกรมโจมตีตัวเดียวกัน เปลี่ยนแค่ dataset	0.9999 ถึง 1.0000
ผลหลัก ข้าม dataset	0.9994 ถึง 1.0000

กติกาก็เขียนไว้ก่อนรันตอบว่า partial ในทุกโมเดล และรายงานตามนั้นโดยไม่ปรับกติกา สาเหตุ มาจาก DoS/DDoS ที่อธิบายไว้ในหัวข้อ 2.4 และจาก Bot ที่ภายใน dataset ของ tree model ได้ 0.89 ถึง 0.94 ซึ่งใกล้เคียง

3. Dataset Importance เทียบกับ Attack Importance

3.1 วิธี

- Dataset importance คือ permutation importance จาก adversarial validation ที่รันไว้แล้ว ไม่ได้คำนวณใหม่
- Attack importance คือ permutation importance ของตัวจำแนก attack ภายในแต่ละ dataset วัดด้วย macro-F1 ใช้โมเดลของ protocol ตัวจริง และทำ preprocess, reseed และ fit เหมือนตอนทำ transfer ทุกอย่าง
- วัดกับสองโมเดล คือ LightGBM ซึ่งเสียหนักที่สุดตอน ablation และ Logistic Regression ซึ่ง ablation ไม่กระทบ และ transfer ทศ 2018 ไป 2017 ได้ดีที่สุด
- ใช้ 60 features ชุดเดียวกัน 5 seeds

วัดสองแบบ เพราะแต่ละแบบมีจุดบอดคนละที่

- permutation importance บอกว่าโมเดลพึ่งคอลัมน์ไหนจริง แต่ถ้ามีหลายคอลัมน์ที่ให้ข้อมูลซ้ำกัน มันจะให้คะแนนต่ำทั้งหมด
- univariate ไม่ใช่โมเดลเลย ฝั่ง dataset ใช้ AUC ของคอลัมน์เดียวระหว่างสอง dataset ฝั่ง attack ใช้ mutual information ระหว่างคอลัมน์กับคลาส attack

feature จะถือว่า "ใช้งาน" ในงานใดงานหนึ่งก็ต่อเมื่อ importance เฉลี่ยเป็นบวก และมากกว่าสองเท่าของส่วนเบี่ยงเบนระหว่าง seed เกณฑ์นี้ตั้งจากความแปรปรวนของการวัดเอง ไม่ได้เลือกด้วยตา

3.2 คำตอบหลัก

คอลัมน์ที่บอก dataset กับคอลัมน์ที่ใช้จำแนก attack เป็นชุดเดียวกันเป็นส่วนใหญ่

rank correlation ระหว่างสองอย่าง (60 features)	2017	2018
univariate แยก dataset ได้ดีแค่ไหน เทียบกับ attack MI	+0.72	+0.77
permutation ของ LightGBM	+0.35	+0.40
permutation ของ Logistic Regression	-0.07	+0.24

ค่าจาก univariate น่าเชื่อถือกว่าเพราะไม่มีปัญหาคอลลิเนียร์ ฝั่ง dataset ของ permutation importance มี 48 จาก 60 features ที่คะแนนเกือบศูนย์เพราะปัญหานี้

ตระกูลขนาด packet อยู่อันดับต้นของทั้งสองด้านพร้อมกัน

feature	AUC คอลลิเนียร์เดียวข้าม dataset	attack MI 2017	attack MI 2018
Packet Length Mean	0.8290	0.4965	0.2633
Average Packet Size	0.8279	0.5282	0.2599
Packet Length Std	0.8105	0.4777	0.2736
Max Packet Length	0.8054	0.4796	0.2747

3.3 บทบาทของแต่ละ feature

มี 18 features ที่แยก dataset ได้เกินระดับ noise อย่างน้อยในหนึ่งคลาส

บทบาท	ตรงกันทั้ง LightGBM และ Logistic Regression
fingerprint อย่างเดียว ตัดทิ้งได้โดยไม่เสีย attack signal	มีตัวเดียว คือ act_data_pkt_fwd
ใช้ทั้งสองอย่าง	min_seg_size_forward, Init_Win_bytes_forward, Init_Win_bytes_backward, Destination Port, Flow IAT Min, Fwd Packet Length Std, Bwd Packet Length Std

อีก 10 ตัวขึ้นกับโมเดล เป็น fingerprint อย่างเดียวสำหรับ LightGBM แต่ Logistic Regression เอาไปใช้จำแนก attack

3.4 ทำไม ablation ทำให้ tree model แย่ลงแต่ Logistic Regression ไม่เป็นไร

feature 10 อันดับแรกที่ ablation ตัดทิ้ง คิดเป็นสัดส่วนเท่าไรของ attack importance

โมเดล	2017	2018
LightGBM	80.8%	78.9%
Logistic Regression	12.6%	31.1%

ตัวหลักคือ Destination Port ซึ่งด้าน dataset อยู่อันดับ 6 แต่เป็น feature ที่ LightGBM ใช้จำแนก attack มากที่สุด ประมาณ 40% ของ importance ทั้งหมด

ข้อจำกัด ที่ K=5 สัดส่วนนี้ของ LightGBM เหลือเพียง 10% ถึง 27% แต่ transfer ของ LightGBM ตกหนักตั้งแต่ K=5 แล้ว จาก 26.2% เหลือ 9.9% importance ภายใน dataset จึงอธิบายการตกได้ส่วนหนึ่ง ไม่ใช่ทั้งหมด

3.5 ข้อค้นพบเพิ่ม: RST Flag Count

	2017	2018
สัดส่วนที่เป็นศูนย์	99.97%	76.3%

ECE Flag Count กระจายตัวแบบเดียวกันทุกอย่าง โมเดลที่เทรนบน 2017 จึงใช้คอลลิเนียร์ไม่ได้เลย (importance เท่ากับศูนย์พอดี) เป็นไปได้ว่าตัวตั้ง feature ของสอง dataset นั้น TCP flag ไม่เหมือนกัน

ข้อนี้ยังไม่ได้ยืนยันกับเอกสารของ CIC

3.6 สรุปข้อ 3

ไม่มีชุด fingerprint ที่แยกออกมาตัดทิ้งได้อย่างสะอาด ซึ่งอธิบายได้ว่าทำไม ablation ในรอบก่อน ถึงไม่ช่วย และทำไมมันทำร้าย tree model มากกว่าโมเดลเชิงเส้น ทางที่เหลือยู่จึงเป็น target adaptation หรือการเทรนร่วมสอง dataset ซึ่งอาจารย์กันไว้ไม่ให้ทำในขั้นนี้

4. ผล Cross-Dataset หลังแก้ Clean-before-Sampling

4.1 สิ่งที่เปลี่ยน

protocol v1 สุ่มคลัง 2018 ให้เหลือ 300,000 แถว ก่อน ทำความสะอาด ส่วน protocol v2 สร้างคลัง 2018 ใหม่จากไฟล์ดิบ 10 ไฟล์ ทำความสะอาดก่อนแล้วค่อยสุ่ม เหมือนที่ฝั่ง 2017 ทำมาตลอด

ทั้งสองรอบใช้ 7 โมเดล 2 วิธีแบ่ง 5 seeds และ schema เดียวกัน สิ่งเดียวที่เปลี่ยนคือลำดับการเตรียมข้อมูลของคลัง 2018 ความต่างทุกอย่างจึงมาจากการแก้

4.2 Asymmetry ยังอยู่ แต่เล็กลง

ค่า recovered % แบ่งตามเวลา คือส่วนของระยะจาก baseline ถึง ceiling ที่ transfer ปิดได้

	v1	v2
2017 ไป 2018 เฉลี่ย	4.2%	5.7%
2018 ไป 2017 เฉลี่ย	44.6%	39.3%
asymmetry เฉลี่ย	40.4 จุด	33.6 จุด
asymmetry median	48.0 จุด	34.3 จุด
โมเดลที่ 2018 ไป 2017 ดีกว่า	7 จาก 7	7 จาก 7
โมเดลที่ดีที่สุดทิศ 2018 ไป 2017	stacking 73.9%	logistic_regression 79.8%

รายโมเดล แบ่งตามเวลา

โมเดล	v1 2017 ไป 2018	v1 2018 ไป 2017	v2 2017 ไป 2018	v2 2018 ไป 2017
catboost	3.4%	26.3%	5.4%	15.9%
lightgbm	7.5%	61.5%	9.9%	52.8%
logistic_regression	4.4%	53.3%	2.8%	79.8%
mlp	3.7%	40.3%	6.1%	39.7%
random_forest	3.6%	6.0%	5.4%	7.2%
stacking	3.6%	73.9%	5.4%	40.2%
xgboost	3.2%	51.2%	5.2%	39.5%

ทิศทางเหมือนเดิมทุกโมเดล และทิศ 2017 ไป 2018 ยังเกือบเท่า baseline แต่ลำดับโมเดลในทิศ 2018 ไป 2017 เปลี่ยนไปมาก stacking ที่เคยดีที่สุดตกจาก 73.9% เหลือ 40.2%

4.3 การแบ่งแบบสุ่มยังซ่อน asymmetry ไว้

การแบ่งแบบสุ่ม	v1	v2
asymmetry เฉลี่ย	0.7 จุด	4.3 จุด
asymmetry median	-0.1 จุด	-0.3 จุด
โมเดลที่ 2018 ไป 2017 ดีกว่า	3 จาก 7	3 จาก 7

ถ้ารันเฉพาะการแบ่งแบบสุ่มจะสรุปว่าสองทิศยากพอกัน ข้อนี้ ไม่เปลี่ยนหลังการแก้

สาเหตุคือการแบ่งแบบสุ่มด้าน ceiling ของ 2017 ขึ้นมาก เมื่อ ceiling สูงขึ้น recovered % ของทุก transfer ที่ทดสอบกับ 2017 ก็ลดลงตาม แม้ transfer จริงจะไม่ได้เปลี่ยน

ผลของการแบ่งแบบสุ่ม ต่อ recovered %	v1	v2
ทิศ 2017 ไป 2018	+10.8 จุด	+5.7 จุด
ทิศ 2018 ไป 2017	-29.0 จุด	-23.5 จุด

4.4 Ceiling และ inflation

	v1	v2
ceiling 2018 แบ่งตามเวลา	0.7236	0.8293
ceiling 2018 แบ่งแบบสุ่ม	0.7481	0.8422
ceiling 2017 แบ่งตามเวลา	0.6495	0.6491
inflation ของ 2018	0.0246	0.0129
inflation ของ 2017	0.2567	0.2559

ceiling ของ 2018 สูงขึ้นมาก เพราะคลังใหม่มีคลาสหายากครบ Web Attack เพิ่มจาก 17 เป็น 859 แถว จึงเรียนรู้ได้ ส่วน inflation ของ 2018 ที่ลดลงครึ่งหนึ่งนั้น 70% ของค่าเดิมมาจาก Web Attack ซึ่งเคยมี test เพียง 5 แถว ไม่ใช่เพราะปิด leakage การ dedup ทำก่อนการแบ่งข้อมูลทั้งสองรอบอยู่แล้ว

4.5 Control ในตัว

คลัง 2017 ไม่ได้ถูกแตะเลย ค่า ceiling จึงควรออกมาเท่าเดิม

	โมเดล
ได้ค่าเท่าเดิมทุกตำแหน่ง	lightgbm, logistic_regression, random_forest
ขยับเล็กน้อย	catboost +0.0018, stacking -0.0110, xgboost -0.0142, mlp +0.0208

สามโมเดลที่ผลคงที่ทุกครั้งให้ค่าเท่าเดิมพอดี ส่วนสี่โมเดลที่ขยับเป็นโมเดลที่ผลแต่ละรอบไม่ ดายตัวอยู่แล้ว ยืนยันว่าความเปลี่ยนแปลงทั้งหมดมาจากฝั่ง 2018

4.6 สรุปข้อ 4

การแก้ไม่ได้ล้มข้อสรุปหลัก asymmetry ยังอยู่ และการแบ่งแบบสุ่มยังช้อนมันไว้เหมือนเดิม แต่ขนาดของ asymmetry ลดลงราว 7 จุด และลำดับโมเดลใน transfer เปลี่ยนไปพอสมควร ตัวเลข stacking 73.9% ที่เคยเป็นผลดีที่สุดของงานเป็น artifact ของคลัง 2018 เวอร์ชันเดิม

5. ข้อจำกัด

1. ช่วงเวลาไม่เท่ากัน ภายใน dataset เทียบกันห่างไม่กี่วัน แต่ข้าม dataset ห่างกันราว 8 เดือน ส่วนหนึ่งของช่องว่างอาจเป็นแค่เวลาที่ผ่านไปนานกว่า ข้อมูลชุดนี้ไม่มีทางแก้ข้อนี้ได้
2. Bot ภายใน dataset ยังอธิบายไม่ได้ tree model ได้ 0.89 ถึง 0.94 เมื่อแบ่งหน้าตาการโจมตี Bot ของปี 2017 ออกเป็นสองครึ่ง ยังไม่ทราบว่าเป็นเพราะ n เล็กหรือหน้าตานั้นมีสองช่วงจริง
3. การวัดความต่างระหว่างวันแบบสะอาดมีแค่สองจุด ทั้งคลัง 2018 มี sub-label เพียงสองตัวที่ครอบคลุมมากกว่าหนึ่งวันโดยมีแถวพอ ส่วนคลัง 2017 ทุก sub-label อยู่ในไฟล์เดียว

4. permutation importance มองไม่เห็นคอลัมน์ที่ซ้ำกัน จึงรายงาน univariate ประกอบเสมอ
5. กลไกของ RST Flag Count ยังไม่ได้ยืนยัน
6. Infiltration ทดสอบไม่ได้ เพราะฝั่ง 2017 มีเพียง 36 แถว

6. ไฟล์ที่ใช้

สคริปต์

สคริปต์	ใช้ในข้อ
scripts/diagnose_av.py	1
scripts/within_dataset_control.py	2
scripts/capture_day_control.py	2
scripts/tool_matched_av.py	2
scripts/dataset_vs_attack_importance.py	3
scripts/compare_protocols.py	4

ข้อมูลใน data/

ไฟล์	ที่มาใน results/crossdataset/	ใช้ในข้อ
diagnostics.json	adversarial_validation/	1
within_dataset_control.json	adversarial_validation/	2
capture_day_control.json	adversarial_validation/	2
tool_matched.json	adversarial_validation/	2
CONTROL_PREREGISTRATION.md	adversarial_validation/	2
feature_roles.csv	importance_comparison/	3
importance_comparison.json	importance_comparison/comparison.json	3
protocol_comparison.json	protocol_comparison/comparison.json	4
per_model_recovered.csv	protocol_comparison/	4
summary_gap_v2.csv	protocol_v2/summary_gap.csv	4
summary_leakage_v2.csv	protocol_v2/summary_leakage.csv	4